COMP315 — AI Software Test & ML/Ops

Project Report

A TensorFlow Extended (TFX) and Apache Airflow ML Pipeline for Bank Marketing Term-Deposit Subscription Prediction

Group 6

Section: SEC.001

PUBLIC PORTFOLIO EDITION

Table of Contents

Table of Contents 2
1. Rationale and Scope 3
2. Pipeline Design 3
  2.1 Component Graph (Steps 1–10) 3
  2.2 Data Preparation and Feature Engineering 4
  2.3 Model Design 4
  2.4 Evaluation and Deployment 5
  2.5 Airflow Orchestration 5
3. Data Validation Report 6
  3.1 StatisticsGen Results 6
  3.2 SchemaGen Results 9
  3.3 ExampleValidator Result 9
4. Model Training and TensorBoard Results 10
5. TFMA Analysis & Fairness 12
  5.1 Overall Metrics 12
  5.2 Slicing by Marital Status 13
  5.3 Fairness Indicator 14
    Lowest-performing slice and hypothesis 17
  5.4 Comparing Two Pipeline Runs 17
6. What-If Tool Analysis 19
  6.1 Contrasting Example 1 20
  6.2 Contrasting Example 2 21
  6.3 Contrasting Example 3 22
  6.4 Fairness Threshold Analysis 22
  6.5 Partial Dependence Analysis 25
7. Model Blessing and Deployment 25
8. Final Reflection 26
  8.1 What did ExampleValidator catch that raw-data inspection would have missed? 26
  8.2 How did TFMA slicing reveal something overall accuracy alone would not? 26
  8.3 What did the What-If Tool reveal about individual predictions? 26
  8.4 What would we change about the pipeline for production deployment? 26
  8.5 Limitations of the current model and training data 27
9. Conclusion 27
10. Assumptions 27
11. References 28
Appendix 1: Meeting Register 29

1. Rationale and Scope

This project was completed for COMP315 (AI Software Test & ML/Ops) with the goal of experiencing how a production machine learning workflow is structured as an automated, repeatable, and auditable sequence of steps, rather than as a one-off notebook script. Group 6 built an end-to-end pipeline using TensorFlow Extended (TFX) and Apache Airflow, applied to the UCI Bank Marketing dataset, to predict whether a bank client will subscribe to a term deposit following a telemarketing campaign.

The scope of the project covers nine TFX components (ExampleGen through Pusher) organized as a ten-step implementation, a TensorFlow Model Analysis (TFMA) notebook that slices model performance across a sensitive or relevant feature, and a What-If Tool (WIT) notebook that inspects individual model predictions through contrasting examples and partial-dependence plots. The intent is not to produce a production-grade marketing model, but to demonstrate correct use of the TFX component graph, TFMA slicing, and WIT interpretability tooling, and to critically discuss what each stage revealed about the data and the model.

The dataset (bank-full.csv, UCI Machine Learning Repository) contains 45,211 records describing the outcome of a Portuguese bank's direct-marketing phone campaigns, with 16 input features (7 numeric, 9 categorical) and a binary target y (1 = client subscribed to a term deposit, 0 = did not).

2. Pipeline Design

The project contains an interactive TFX walkthrough using InteractiveContext and an Airflow DAG in bank_marketing_dag.py. Each constructs the main TFX components separately. The additional create_pipeline() function in pipeline.py is not imported by the Airflow DAG and uses different evaluation thresholds. The Airflow DAG is manually triggered rather than scheduled. The submitted walkthrough is provided as a Python export of a Jupyter notebook.

2.1 Component Graph (Steps 1–10)

Step	Component	Purpose / configuration
1. ExampleGen	CsvExampleGen	Reads all CSVs under data/bank_marketing/, splits rows into train/eval (2:1 default), converts to TFRecord-backed tf.Examples.
2. StatisticsGen	StatisticsGen	Computes per-feature statistics (min/max/mean, missing %, histograms) for both splits.
3. SchemaGen	SchemaGen	Infers feature types, ranges, and categorical vocabularies from the statistics.
4. ExampleValidator	ExampleValidator	Compares live statistics against the schema and flags anomalies (missing values, unexpected types, out-of-range values).
5. Transform	Transform	Applies preprocessing_fn (transform_module.py): z-score scaling for numeric features, decile bucketing of age, and vocabulary lookup for categorical features.

Step	Component	Purpose / configuration
6. Trainer	Trainer	Runs run_fn (trainer_module.py): builds and trains a Keras binary classifier, logs to TensorBoard, exports a SavedModel with a serving signature.
7. Resolver	LatestBlessedModelStrategy	Looks up the most recently blessed model in ML Metadata to serve as the Evaluator's baseline.
8. Evaluator	Evaluator + TFMA	Computes BinaryAccuracy, AUC, Precision, Recall, ExampleCount overall and sliced by marital status; blesses the model only if it clears the configured thresholds.
9. Pusher	Pusher	Copies the trained model to the serving directory only if the Evaluator blessed it.
10. Pipeline fn	create_pipeline()	Separate pipeline functions construct the TFX component graph with a SQLite metadata store. The Airflow DAG defines its own function.

2.2 Data Preparation and Feature Engineering

The original dataset uses semicolons (;) and stores the target y as yes or no. A one-time preparation step converts it into a comma-separated file and changes y to 1 for subscription and 0 for no subscription. The cleaned file is saved as bank_data.csv, which CsvExampleGen divides into training and evaluation data.

Next, the Transform component prepares the features for the neural network. Numerical features are scaled to z-scores so features with large values, such as balance, do not dominate smaller-valued features. Age is also divided into 10 buckets to help the model learn patterns among age groups. Finally, the nine categorical features are converted from text into integer vocabulary IDs so the neural network can process them.

2.3 Model Design

The final trainer_module.py builds a feed-forward Keras neural network with 15,599 trainable parameters. Seven scaled numerical features are combined with embeddings created for the age buckets and categorical features. The combined inputs pass through three dense layers containing 128, 64, and 16 neurons. Each layer uses the ReLU activation function and is followed by a dropout layer with a rate of 0.2 to reduce overfitting. The final layer contains one sigmoid unit that predicts the probability that a customer will subscribe to a term deposit.

The model uses the Adam optimizer with a learning rate of 0.0005 and binary cross-entropy as its loss function. It is trained for 10 epochs with a batch size of 128. Since subscription examples are less common, class weights of 1.0 for non-subscribers and 4.0 for subscribers are applied. Binary accuracy and AUC are tracked during training and evaluation.

The Trainer also records training results in TensorBoard and exports the trained model in TensorFlow SavedModel format. The exported serving function includes the Transform graph, allowing the deployed model to accept raw customer records and apply the same preprocessing used during training.

2.4 Evaluation and Deployment

The Evaluator component uses TensorFlow Model Analysis (TFMA) to assess the trained model on evaluation examples, overall and across marital-status groups including married, single, and divorced customers. Sliced evaluation helps identify performance differences that overall metrics may hide.

The evaluation measures BinaryAccuracy, AUC, Precision, Recall, and ExampleCount. To be approved, the model must achieve a BinaryAccuracy of at least 0.70. Its accuracy must also not decrease by more than 0.01 compared with the previously approved model. The Resolver component retrieves this previous model and provides it as the baseline for comparison.

During the first pipeline run, no baseline model is available, so baseline-change validation is skipped; the model must still satisfy the absolute accuracy threshold. In later runs, a new model must satisfy both applicable evaluation requirements before it can be blessed. The Pusher component exports only the blessed model to the serving directory.

2.5 Airflow Orchestration

The bank_marketing_dag.py file uses AirflowDagRunner to run each TFX component as an Airflow task. Airflow manages task dependencies, displays their status, and provides logs for finding errors. The DAG is manually triggered using schedule_interval=None, which is suitable for testing and project demonstrations.

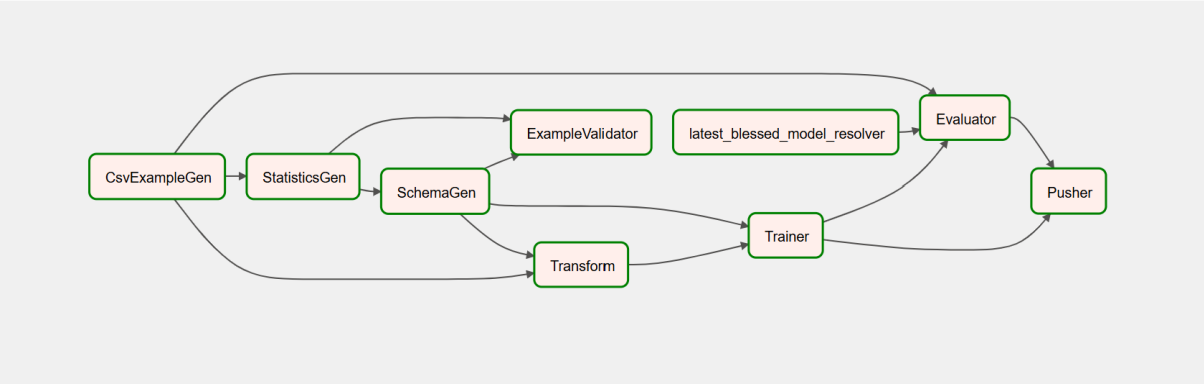


Figure 2.1: Airflow Graph View showing all Bank Marketing TFX pipeline tasks completed successfully.

```
}
. Setting to DenseTensor.
[2026-08-10, 23:13:52 UTC] {save.py:276} WARNING - Found untraced functions such as _update_step_xla while saving (showing 1 of 1). These functions will not be directly callable after loading.
[2026-08-10, 23:13:53 UTC] {builder_impl.py:798} INFO - Assets written to: /
[2026-08-10, 23:13:53 UTC] {executor.py:187} INFO - Training complete. Model written to
[2026-08-10, 23:13:53 UTC] {base_component_launcher.py:212} INFO - Running publisher for Trainer
[2026-08-10, 23:13:53 UTC] {metadata_store.py:103} INFO - MetadataStore with DB connection initialized
[2026-08-10, 23:13:53 UTC] {python.py:175} INFO - Done. Returned value was: None
[2026-08-10, 23:13:53 UTC] {taskinstance.py:1288} INFO - Marking task as SUCCESS. dag_id=bank_marketing_pipeline, task_id=Trainer, execution_date=20260811T031005, start_date=20260811T031208, end_date=
[2026-08-10, 23:13:53 UTC] {local_task_job.py:154} INFO - Task exited with return code 0
[2026-08-10, 23:13:54 UTC] {local_task_job.py:264} INFO - 1 downstream tasks scheduled from follow-on schedule check
```

Figure 2.2: Airflow Trainer task log confirming that model training completed successfully with return code 0.

The successful DAG and Trainer log confirm that the complete pipeline executed in the correct order and that the model-training task finished without errors.

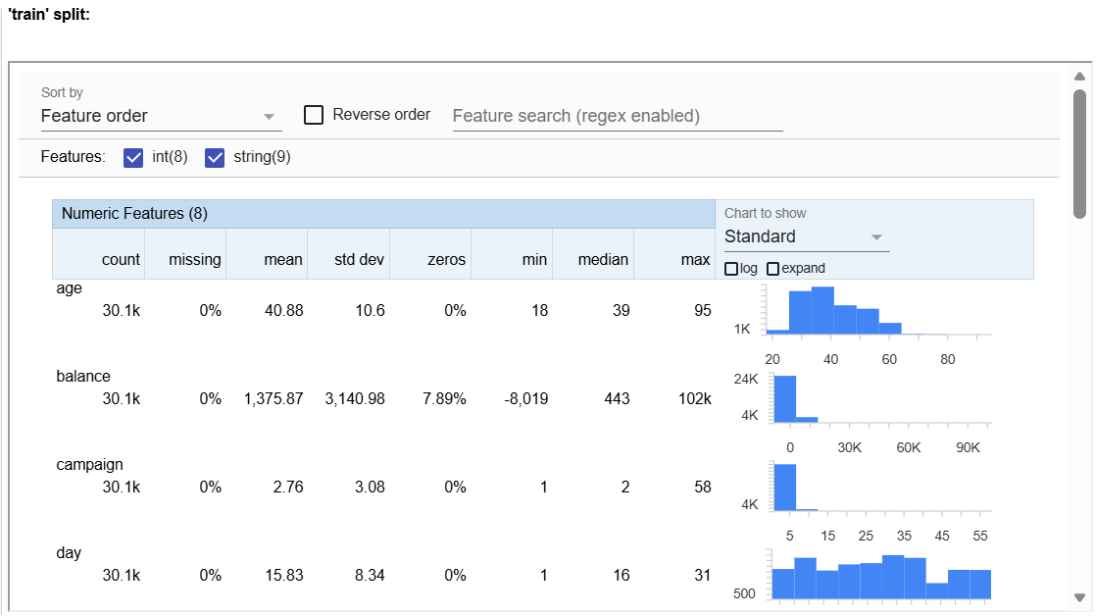
3. Data Validation Report

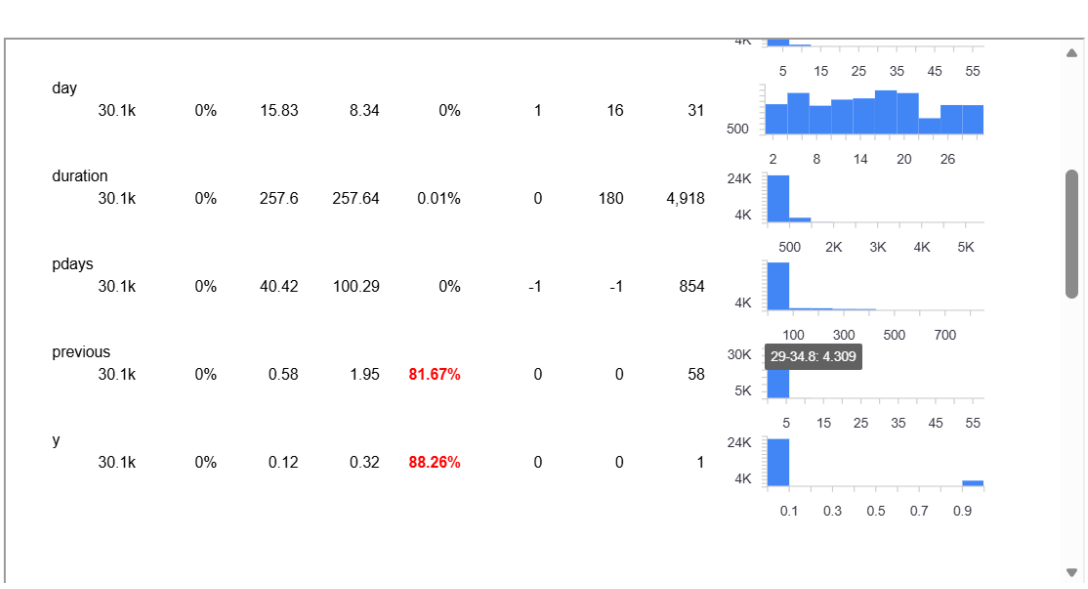
This section summarizes the results produced by StatisticsGen, SchemaGen, and ExampleValidator for the training and evaluation datasets.

3.1 StatisticsGen Results

CsvExampleGen divided 45,211 records into 30,105 training examples and 15,106 evaluation examples. StatisticsGen reported no missing values across the 17 columns. The recorded TFMA result contains 15,088 examples. The available source and screenshots do not establish the cause of this count difference. About 88% of records have y = 0 and 12% have y = 1, so accuracy alone is insufficient for assessing performance.

Several features have uneven distributions. balance contains large positive and negative values, while campaign ranges from a few contacts to a maximum of 58. The categorical features contact and poutcome also contain many unknown values. In addition, duration is strongly related to subscription but is only known after a call, which limits its use for predicting customers before contact.





'train' split:



'train' split:

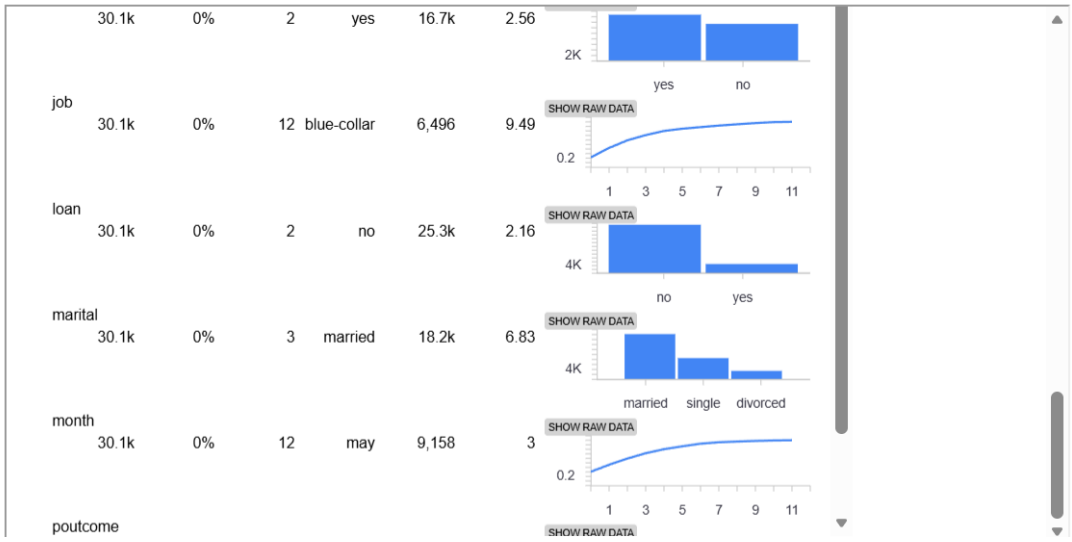
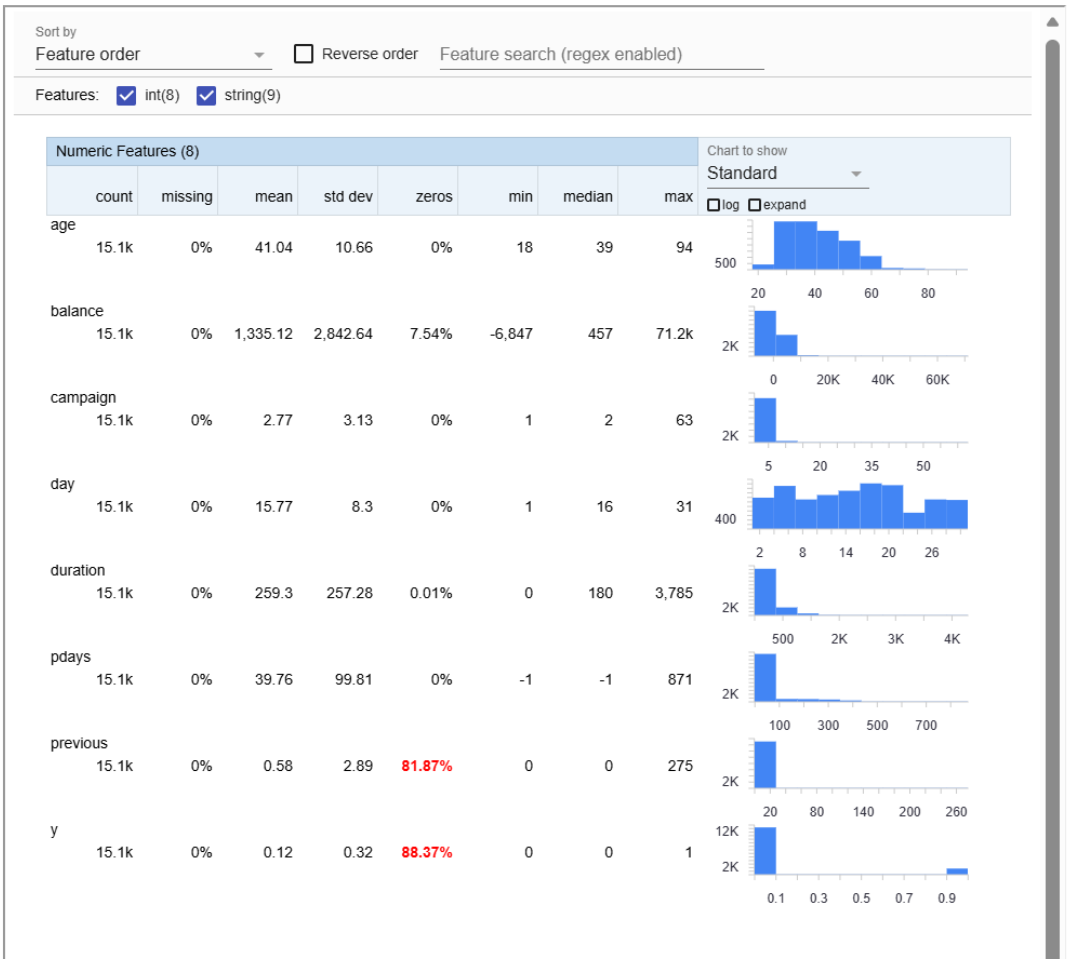
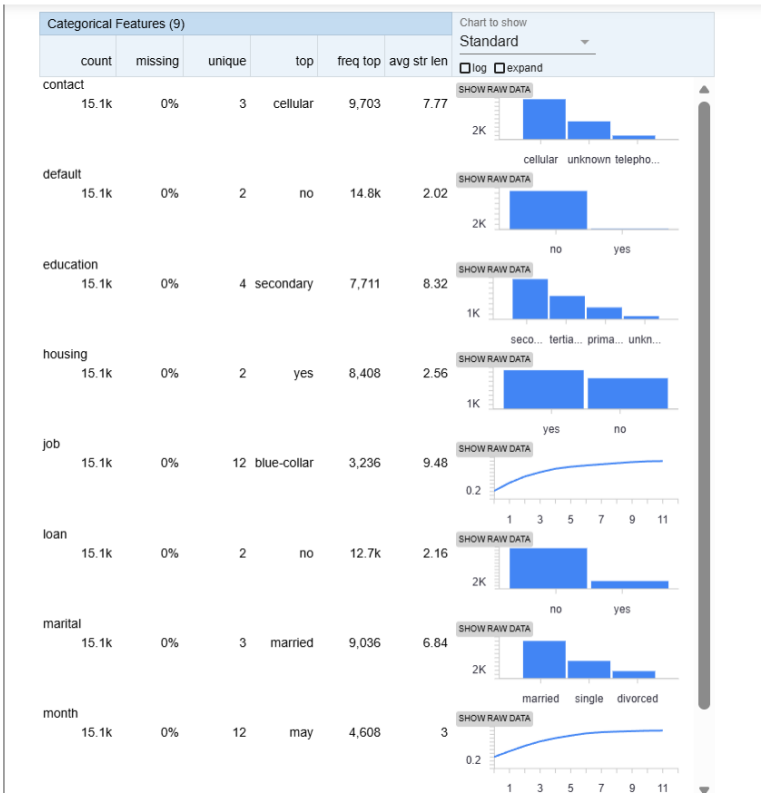


Figure 3.1 (a–d): StatisticsGen results showing numerical and categorical feature statistics for the training and evaluation splits.

'eval' split:


Sort by
Feature order
Reverse order
Feature search (regex enabled)
Features: int(8) string(9)
Numeric Features (8)
Chart to show
Standard
log expand
count missing mean std dev zeros min median max
age 15.1k 0% 41.04 10.66 0% 18 39 94
balance 15.1k 0% 1,335.12 2,842.64 7.54% -6,847 457 71.2k
campaign 15.1k 0% 2.77 3.13 0% 1 2 63
day 15.1k 0% 15.77 8.3 0% 1 16 31
duration 15.1k 0% 259.3 257.28 0.01% 0 180 3,785
pdays 15.1k 0% 39.76 99.81 0% -1 -1 871
previous 15.1k 0% 0.58 2.89 81.87% 0 0 275
y 15.1k 0% 0.12 0.32 88.37% 0 0 1


Categorical Features (9)
Chart to show
Standard
log expand
count missing unique top freq top avg str len
contact 15.1k 0% 3 cellular 9,703 7.77
default 15.1k 0% 2 no 14.8k 2.02
education 15.1k 0% 4 secondary 7,711 8.32
housing 15.1k 0% 2 yes 8,408 2.56
job 15.1k 0% 12 blue-collar 3,236 9.48
loan 15.1k 0% 2 no 12.7k 2.16
marital 15.1k 0% 3 married 9,036 6.84
month 15.1k 0% 12 may 4,608 3
SHOW RAW DATA

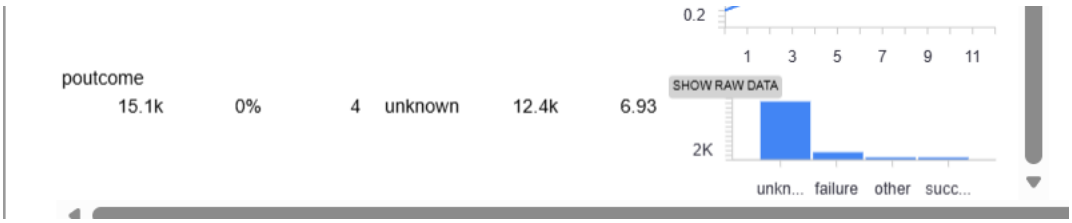


Figure 3.2 (a–c): StatisticsGen numerical and categorical feature statistics for the evaluation split.

The training and evaluation splits show similar distributions, but the target remains strongly imbalanced. The related features previous and pdays describe different aspects of earlier contact: the number of previous contacts and the number of days since a previous contact. Several numerical features, especially balance and campaign, are highly skewed and contain extreme values. In addition, duration may cause data leakage because it is only known after the marketing call; therefore, it would not be available for a real pre-call prediction. These issues are considered during preprocessing, evaluation, and discussion of the model's limitations.

3.2 SchemaGen Results

SchemaGen inferred a schema containing 17 required features, consistent with the 0% missing values reported by StatisticsGen. The seven numerical features and the target label y were identified as integers, while the nine categorical features were identified as strings with fixed vocabularies. These vocabulary sizes are compatible with the embedding layers defined in trainer_module.py.

However, values such as unknown are treated as valid categories rather than missing data. Therefore, ExampleValidator does not flag the high proportion of unknown values in features such as poutcome. Although these values are valid according to the schema, deciding whether they reduce the usefulness of the data requires further interpretation by the project team.

3.3 ExampleValidator Result

ExampleValidator found no anomalies in either the training or evaluation split. The notebook's additional health check therefore reported: **"Data pipeline is HEALTHY: no anomalies found in any split."** This confirms that both splits satisfy the inferred schema.

```
[7]: example_validator = ExampleValidator(
         statistics=statistics_gen.outputs["statistics"],
         schema=schema_gen.outputs["schema"],
     )
     context.run(example_validator)

     context.show(example_validator.outputs["anomalies"])
```

'train' split:

No anomalies found.

'eval' split:

No anomalies found.

Figure 3.3: ExampleValidator results showing no anomalies in the training and evaluation splits.

4. Model Training and TensorBoard Results

During the interactive walkthrough, the model was trained for 10 epochs using 100 training steps and 20 evaluation steps. The full Airflow pipeline used the larger configuration of 1,000 training steps and 200 evaluation steps defined in bank_marketing_dag.py. The Trainer recorded the training and validation loss, binary accuracy, and AUC.

Epoch	Train loss	Train acc.	Train AUC	Val loss	Val acc.	Val AUC
1	0.474	0.9428	0.909	0.234	0.914	0.894
2	0.505	0.8952	0.907	0.341	0.858	0.923
3	0.780	0.8011	0.857	0.470	0.776	0.907
4	1.053	0.6712	0.837	0.521	0.749	0.945
5	0.771	0.7617	0.900	0.242	0.891	0.915
6	0.318	0.9128	0.957	0.210	0.902	0.950
7	0.493	0.8634	0.924	0.211	0.910	0.962
8	0.777	0.7722	0.887	0.502	0.744	0.940
9	0.924	0.7167	0.878	0.414	0.799	0.935
10	0.542	0.8377	0.936	0.250	0.886	0.945

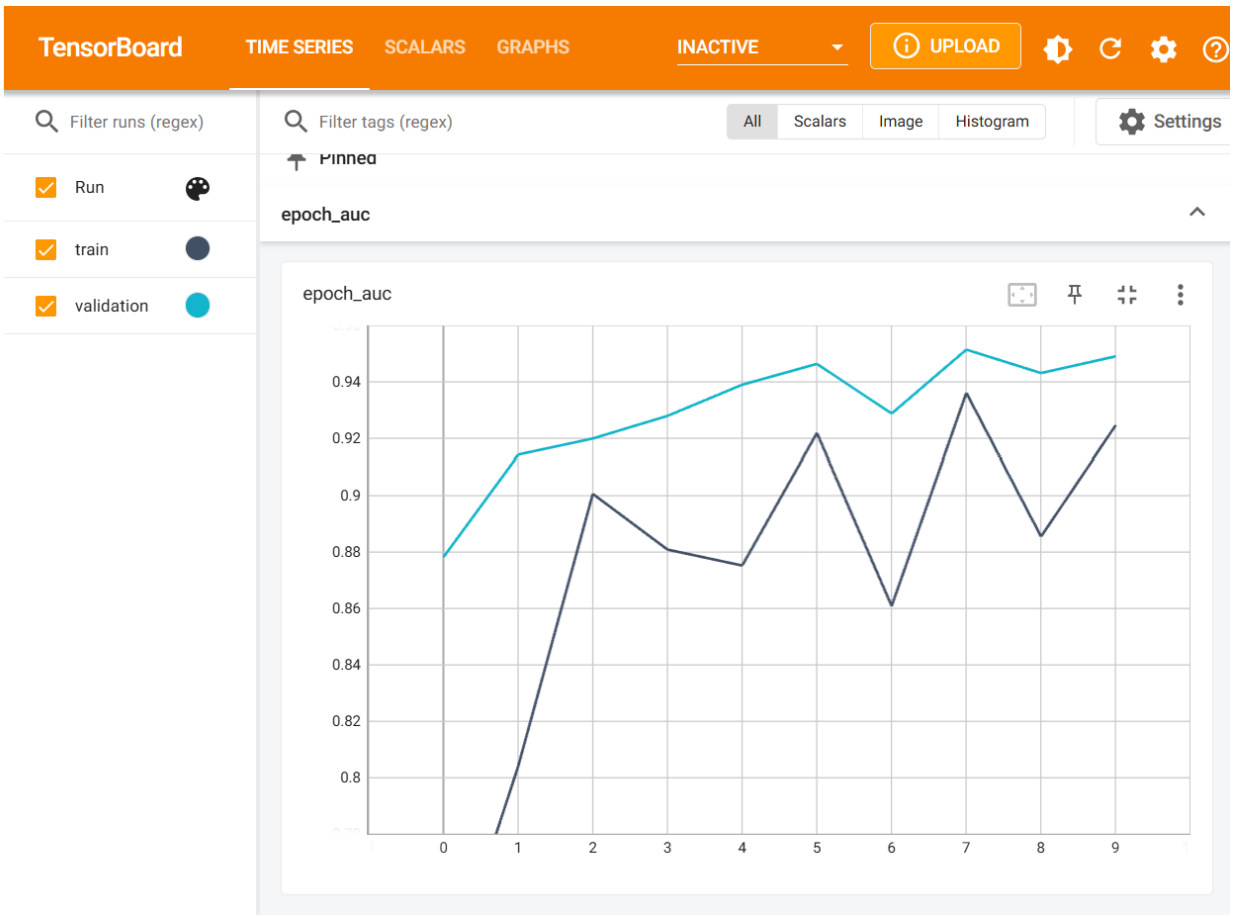


Figure 4.1: TensorBoard training and validation AUC across 10 epochs.

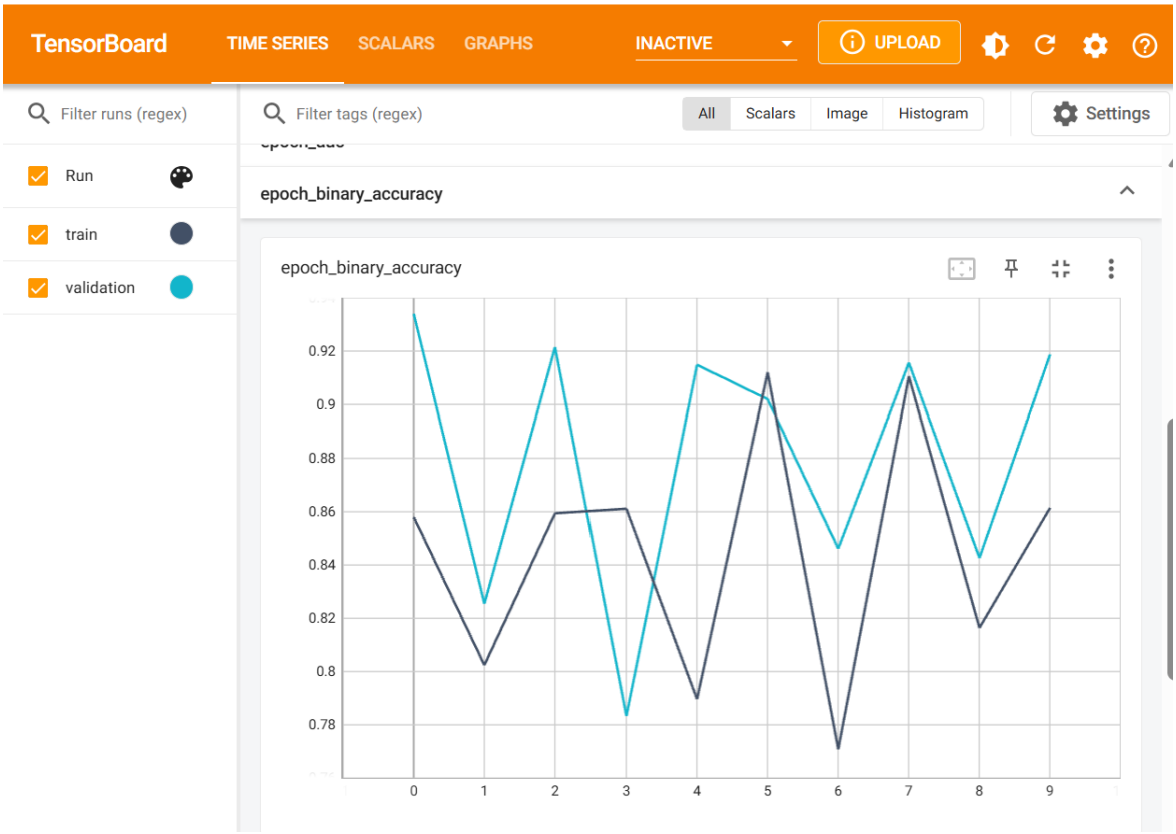


Figure 4.2: TensorBoard training and validation binary accuracy across 10 epochs.

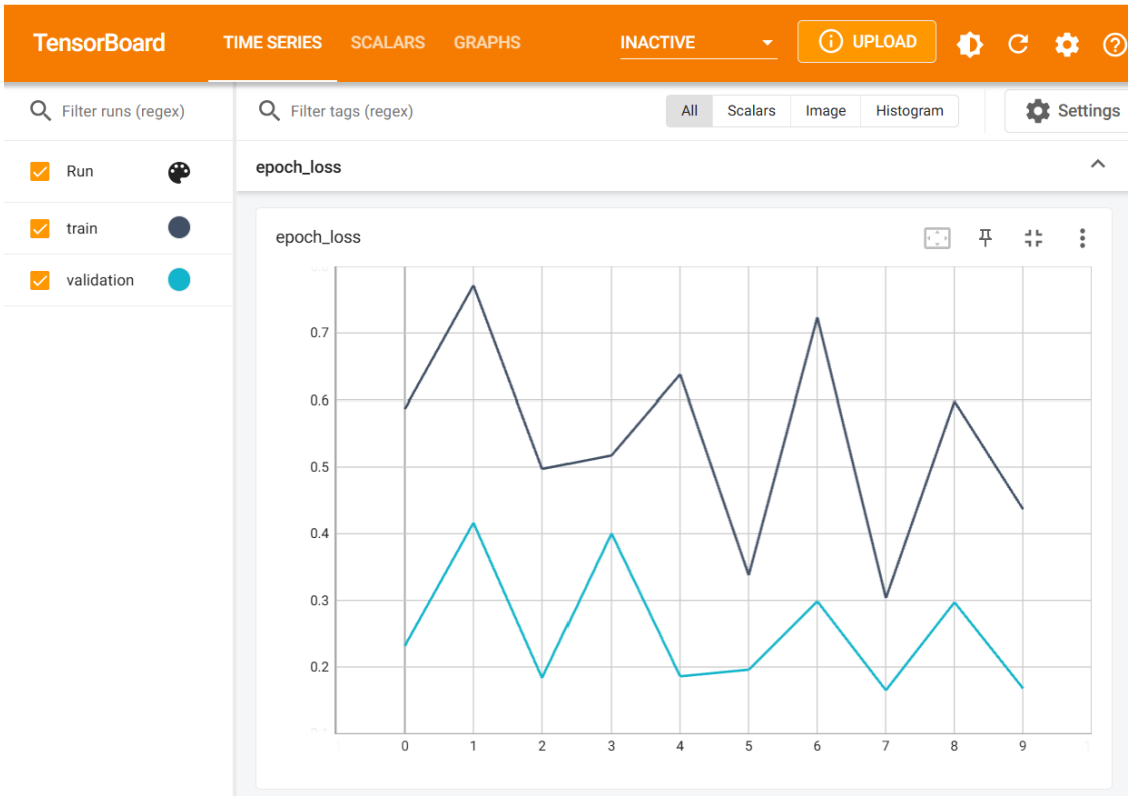


Figure 4.3: TensorBoard training and validation loss across 10 epochs.

For the interactive 100-training-step and 20-evaluation-step walkthrough, TensorBoard shows that validation AUC generally improved across the 10 epochs, while accuracy and loss fluctuated. Its final validation AUC was approximately 0.95. This is an epoch-level metric from the smaller interactive run and should not be confused with the final Model 2 TFMA AUC of 0.9145 from the pipeline evaluation. The fluctuations may be related to class weighting and the smaller interactive configuration.

5. TFMA Analysis & Fairness

5.1 Overall Metrics

TFMA evaluated the model on 15,088 evaluation examples. The model achieved a BinaryAccuracy of 0.8791, an AUC of 0.9145, a Precision of 0.4864, and a Recall of 0.7144. The high AUC indicates that the model distinguishes subscribers from non-subscribers well. Recall shows that it identified approximately 71% of actual subscribers, while the lower precision means that approximately 49% of its positive predictions were correct. Because the dataset is strongly imbalanced, AUC, precision, and recall provide more meaningful information than accuracy alone.

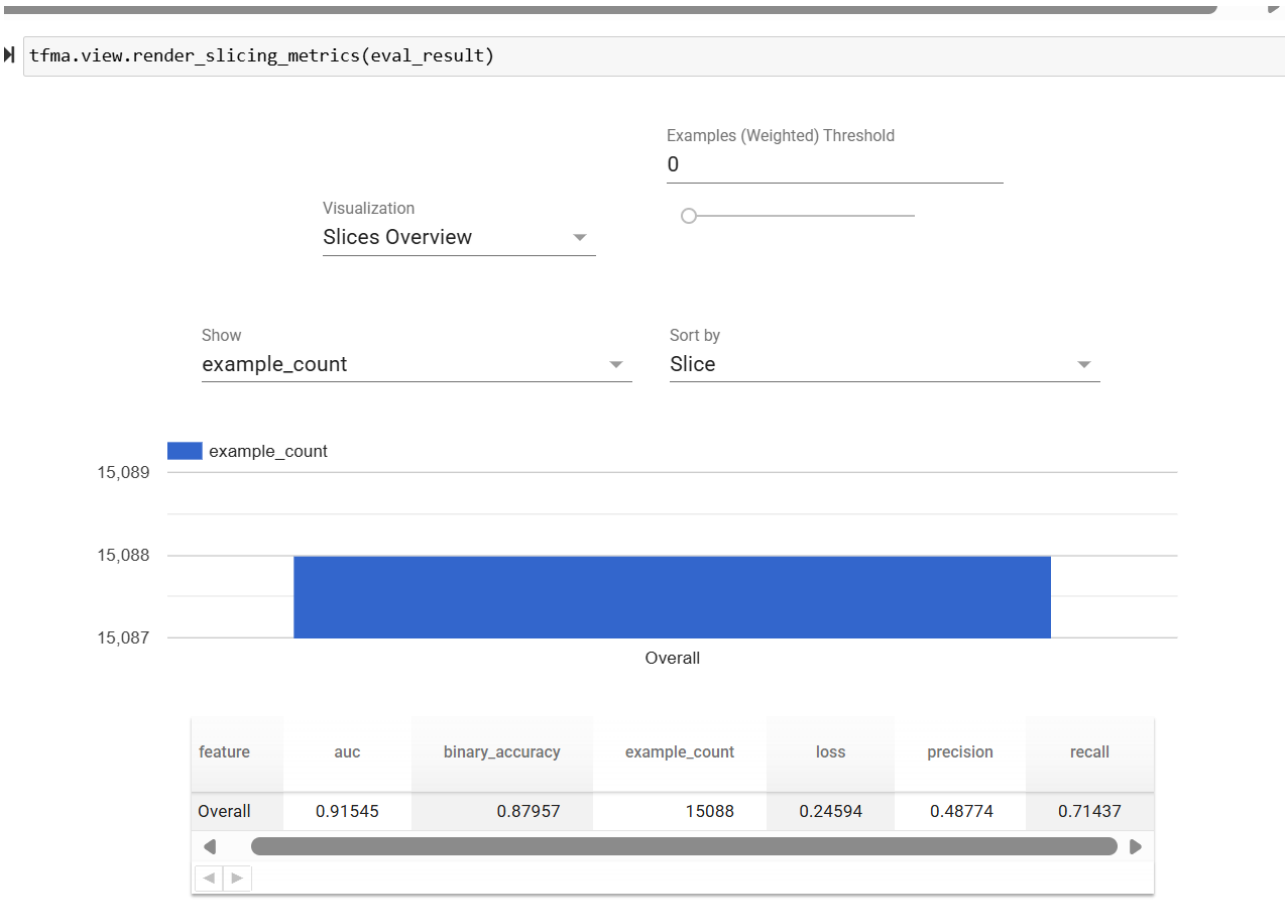


Figure 5.1: TFMA overall evaluation metrics for the final model.

5.2 Slicing by Marital Status

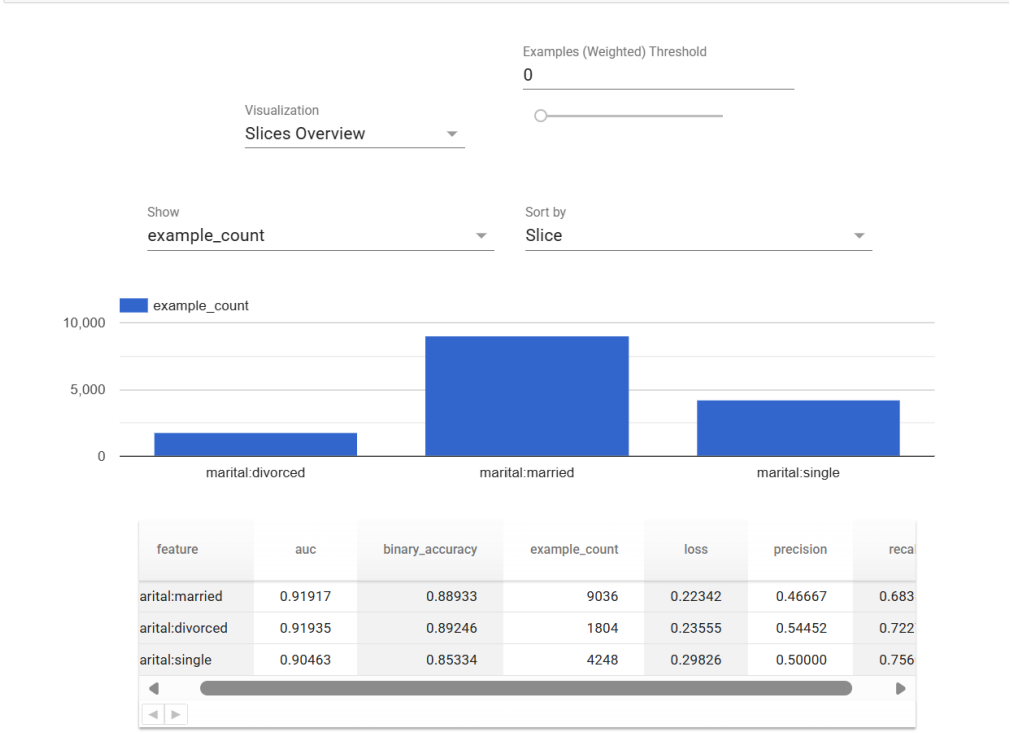


Figure 5.2: TFMA Slices Overview showing evaluation metrics for married, single, and divorced customer groups.

TFMA evaluated the final Model 2 separately for married, single, and divorced customers. Married customers achieved an accuracy of 0.8858 and an AUC of 0.9176 across 9,036 examples. Single customers had the lowest accuracy and AUC, at 0.8595 and 0.9043 across 4,248 examples. Divorced customers achieved an accuracy of 0.8919 and an AUC of 0.9183 across 1,804 examples.

Overall, performance was similar across the groups, although results for the smaller divorced group should be interpreted carefully because its sample size was lower.

5.3 Fairness Indicator

The TFMA Fairness Indicators dashboard was used to compare accuracy, AUC, precision, and recall across the married, single, and divorced groups.

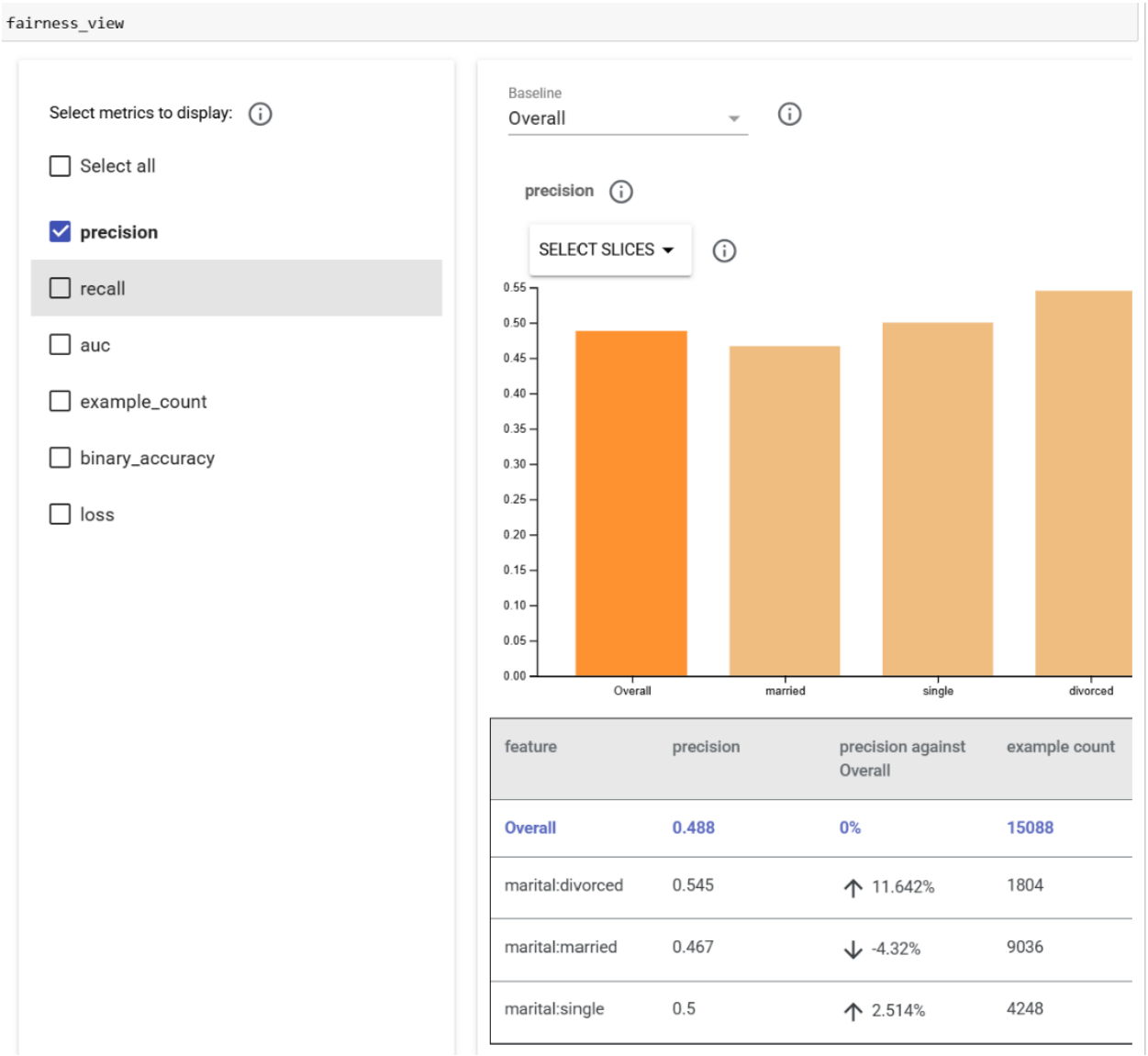


Figure 5.3(a): Precision

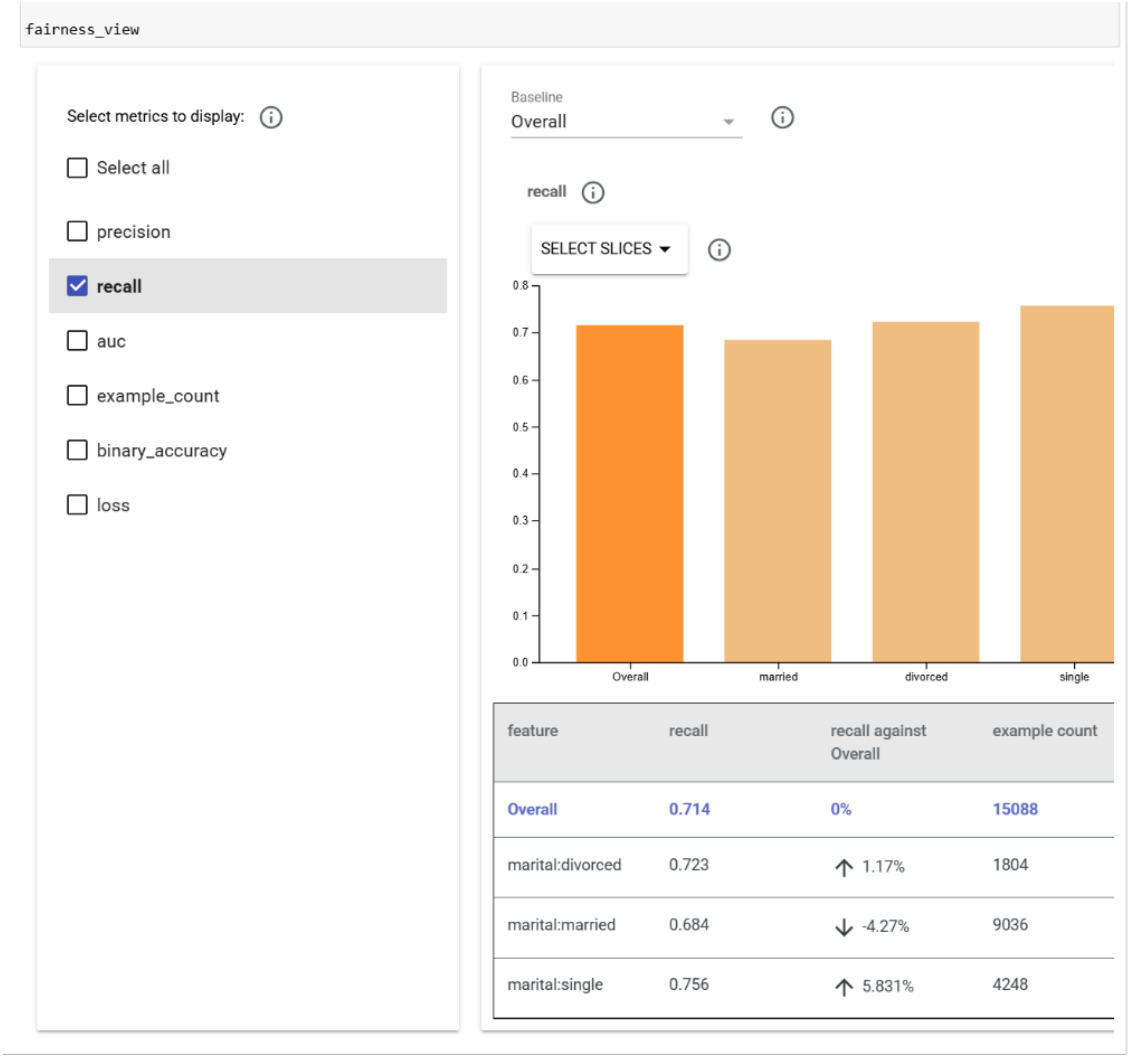


Figure 5.3(b): Recall

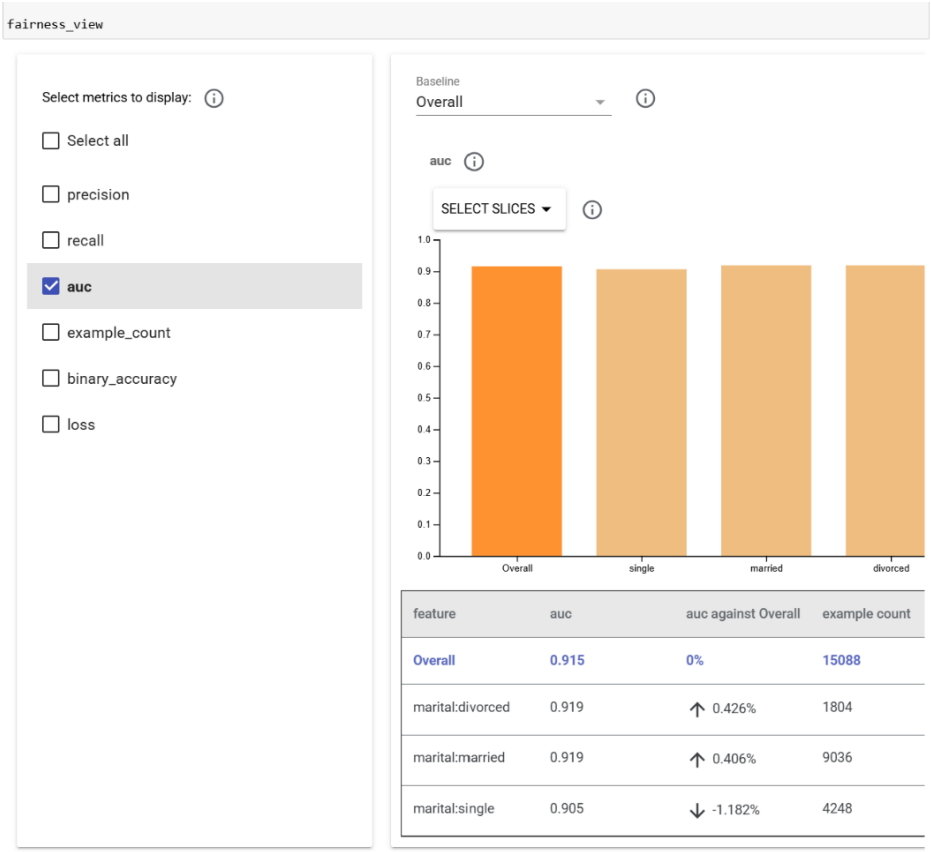


Figure 5.3(c): AUC

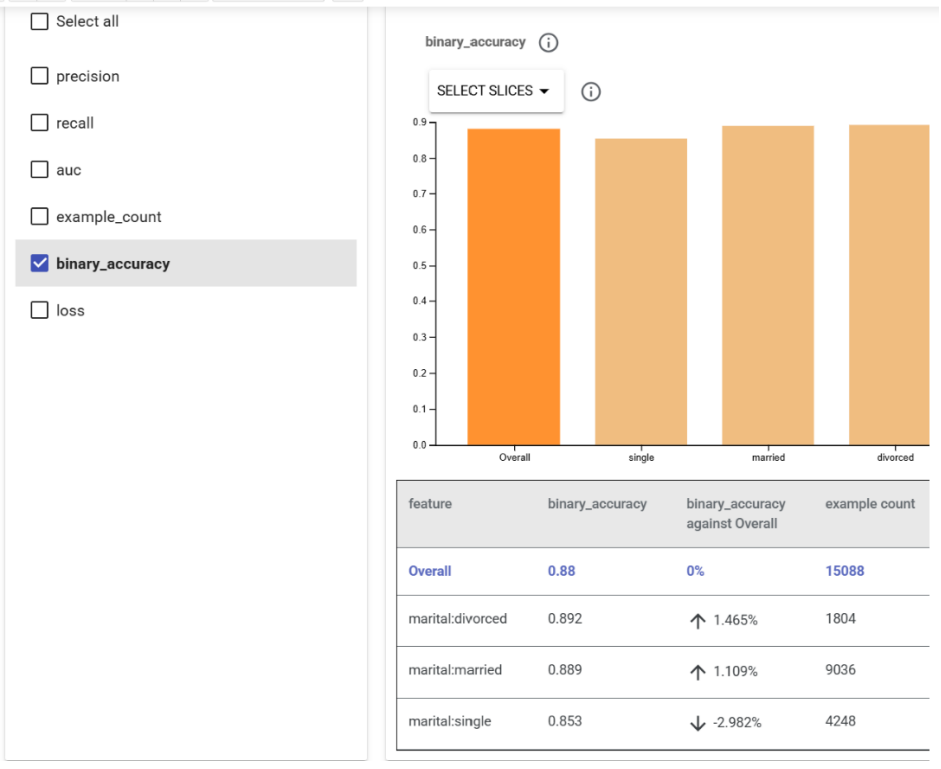


Figure 5.3(d): Binary accuracy

The Fairness Indicators dashboard compared the final model's performance across marital-status groups. Single customers had the lowest accuracy (0.8595) and AUC (0.9043), but the highest recall (0.7480). Precision ranged from 0.4561 for married customers to 0.5430 for divorced customers.

The differences between groups were moderate and did not show a severe performance gap. However, the groups have different sample sizes, so these results should be interpreted carefully and monitored in future model runs.

Lowest-performing slice and hypothesis

Married customers had the lowest precision (0.4561), while single customers had the lowest accuracy (0.8595) and AUC (0.9043). Therefore, no group was consistently the weakest across every metric. These differences may result from variations in group size, feature distributions, or subscription rates. Future work could examine these distributions and test feature interactions or slice-aware reweighting if reducing performance differences becomes a project requirement.

5.4 Comparing Two Pipeline Runs

Two model configurations were evaluated using the same evaluation dataset. Model 1 represents the baseline, while Model 2 represents the final configuration.

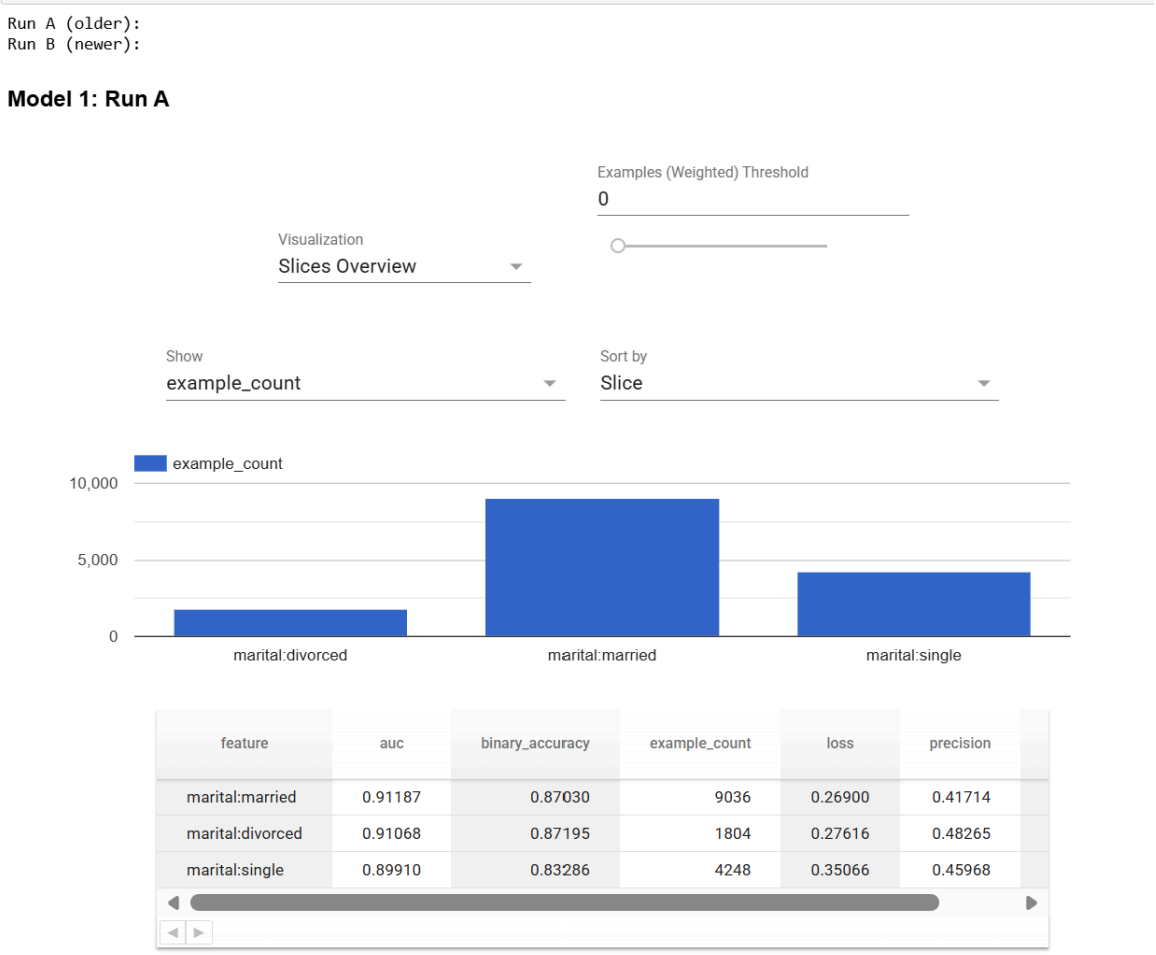
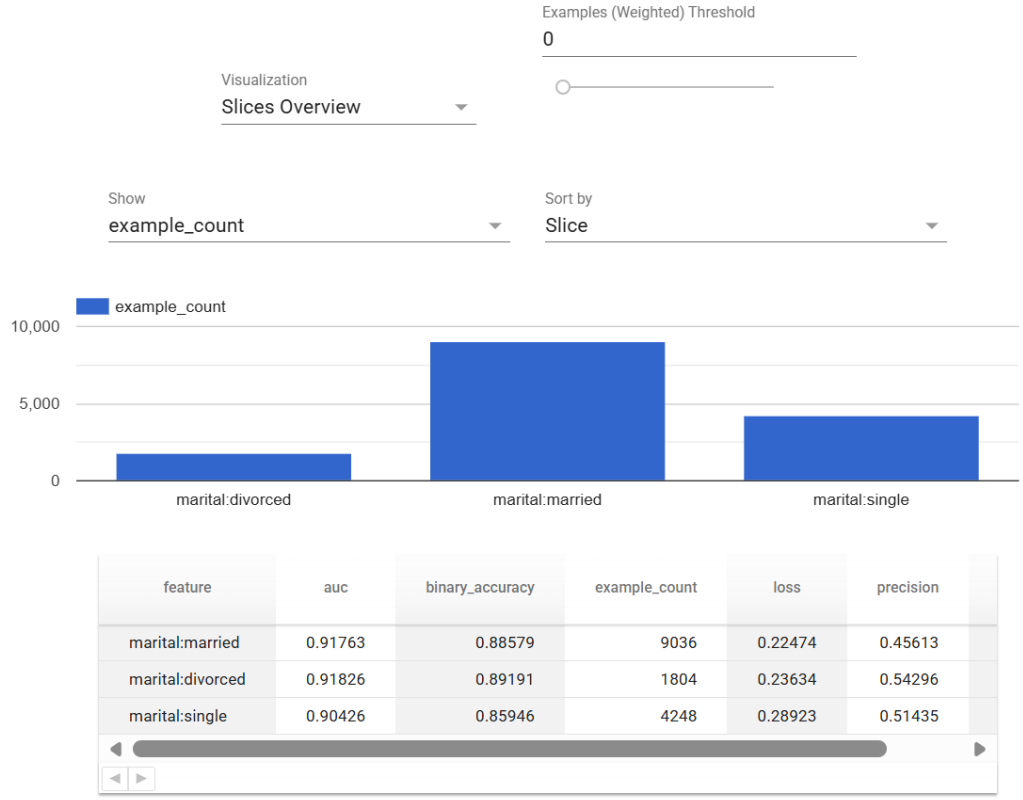


Figure 5.4: TFMA evaluation results for Model 1 (baseline).

Model 2: Run B



feature	auc	binary_accuracy	example_count	loss	precision
marital:married	0.91763	0.88579	9036	0.22474	0.45613
marital:divorced	0.91826	0.89191	1804	0.23634	0.54296
marital:single	0.90426	0.85946	4248	0.28923	0.51435

Figure 5.5: TFMA evaluation results for Model 2 (final model).

TFMA comparison of Model 1 and Model 2

	Slice	Model 1 Example Count	Model 2 Example Count	Model 1 Accuracy	Model 2 Accuracy	Accuracy Change	Model 1 AUC	Model 2 AUC	AUC Change	Model 1 Precision	Model 2 Precision	Model 1 Recall	Model 2 Recall
0	Overall	15088.0	15088.0	0.8600	0.8791	0.0192	0.9084	0.9145	0.0060	0.4396	0.4864	0.7446	0.7144
1	marital=married	9036.0	9036.0	0.8703	0.8858	0.0155	0.9119	0.9176	0.0058	0.4171	0.4561	0.7212	0.6905
2	marital=divorced	1804.0	1804.0	0.8720	0.8919	0.0200	0.9107	0.9183	0.0076	0.4826	0.5430	0.6955	0.7182
3	marital=single	4248.0	4248.0	0.8329	0.8595	0.0266	0.8991	0.9043	0.0052	0.4597	0.5143	0.7961	0.7480

Table 5.1: Comparison of overall and marital-slice metrics for Models 1 and 2.

Two model configurations were evaluated using the same 15,088 examples. Model 2 improved overall accuracy from 0.8600 to 0.8791, AUC from 0.9084 to 0.9145, and precision from 0.4396 to 0.4864. Recall decreased slightly from 0.7446 to 0.7144, meaning Model 2 produced more reliable positive predictions but missed slightly more subscribers.

Model 2 also improved accuracy, AUC, and precision across all three marital-status groups. The largest accuracy improvement occurred for single customers, increasing from 0.8329 to 0.8595. Overall, Model 2 provided a better balance of performance and was selected as the final model.

6. What-If Tool Analysis

The What-If Tool was loaded with 500 evaluation examples and the pushed SavedModel. The team compared individual examples and changed feature values to observe how predictions changed.

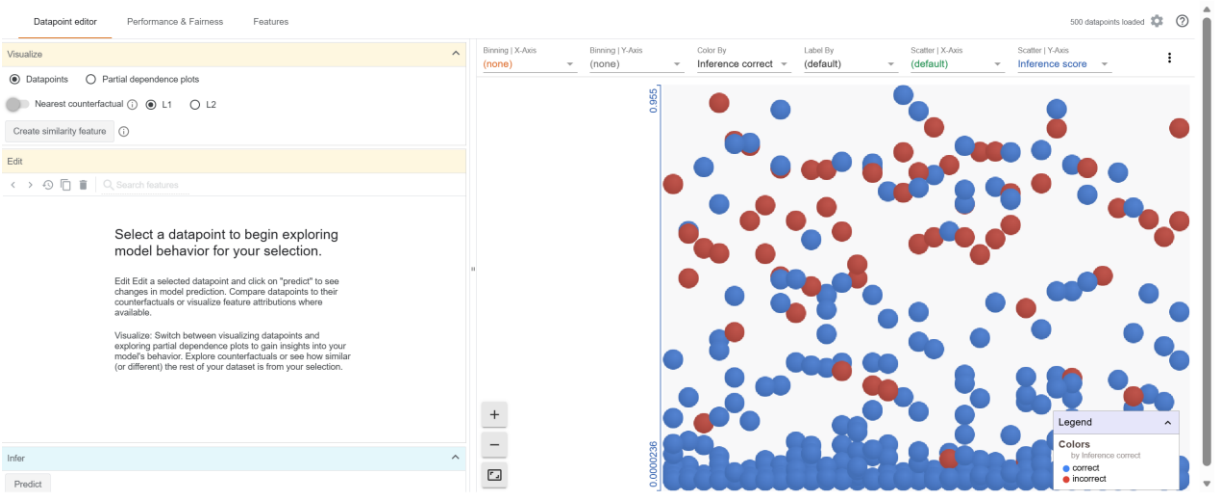


Figure 6.1: What-If Tool overview showing predicted classes and inference scores for 500 evaluation examples.

6.1 Contrasting Example 1



Figure 6.2a–b: Contrasting-example comparison between Datapoints 324 and 194.

Datapoint 324 was incorrectly predicted “yes” at 0.637, while Datapoint 194 was correctly predicted “no” at 0.979. Several customer and campaign features differed, so the prediction change cannot be linked to one feature alone. Blue indicates correct predictions and red indicates incorrect predictions.

6.2 Contrasting Example 2



Figure 6.3a–b: Contrasting-example comparison between Datapoints 46 and 395.

Datapoint 46 was correctly predicted “no” at 0.953, while Datapoint 395 was incorrectly predicted “yes” at 0.501. Differences included age, balance, call duration, job, and campaign day. Since several features changed, the prediction difference cannot be linked to one feature alone.

6.3 Contrasting Example 3



Figure 6.4a–b: Contrasting-example comparison between Datapoints 173 and 302.

Datapoint 173 was correctly predicted “yes” at 0.777, while Datapoint 302 was incorrectly predicted “no” at 0.848. Differences included balance, education, job, previous contacts, and campaign outcome. These combined changes produced a strong shift in the model’s prediction.

6.4 Fairness Threshold Analysis

The Performance & Fairness view compared the model across marital-status groups using a classification threshold of 0.5.

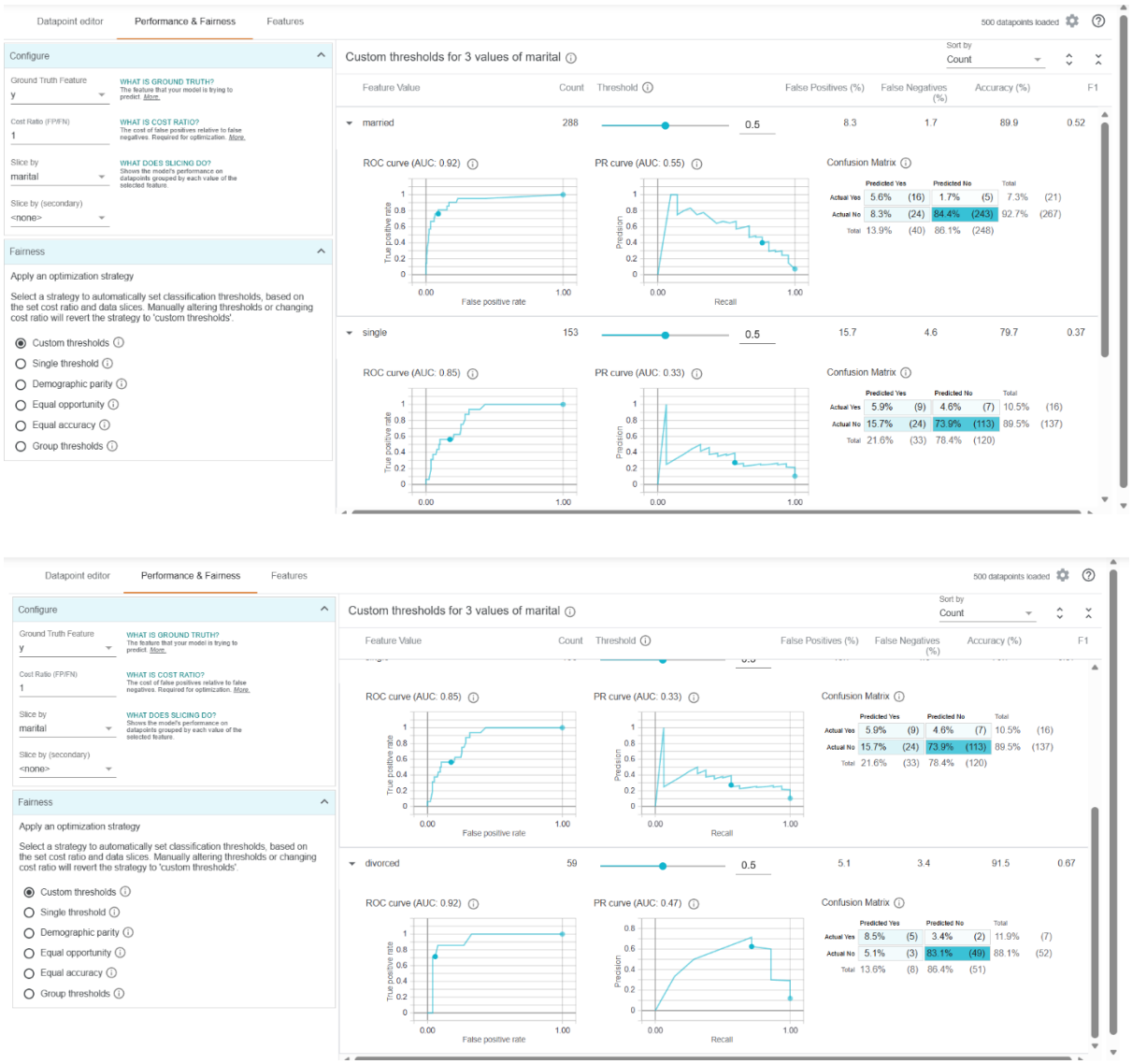


Figure 6.5a–b: What-If Tool fairness analysis for married, single, and divorced customers.

Married and divorced customers achieved approximately 89.9% and 91.5% accuracy, while the single group had lower accuracy at 79.7%. The single group also had the lowest ROC AUC (0.85) and F1 score (0.37). However, these results use only 500 sampled examples, and the divorced group contains only 59 examples, so the differences should be interpreted cautiously.

```
Fairness threshold analysis by marital status
```

	Feature	Threshold	Slice	Examples	Accuracy	False Positive Rate	False Negative Rate	Positive Prediction Rate
16	marital	0.30	Overall	500	0.8360	0.1579	0.2273	0.2120
17	marital	0.30	divorced	59	0.8644	0.1346	0.1429	0.2203
18	marital	0.30	married	288	0.8750	0.1199	0.1905	0.1701
19	marital	0.30	single	153	0.7516	0.2409	0.3125	0.2876
32	marital	0.50	Overall	500	0.8820	0.0987	0.3182	0.1500
33	marital	0.50	divorced	59	0.9153	0.0577	0.2857	0.1356
34	marital	0.50	married	288	0.9167	0.0712	0.2381	0.1215
35	marital	0.50	single	153	0.8039	0.1679	0.4375	0.2092
48	marital	0.70	Overall	500	0.9160	0.0439	0.5000	0.0840
49	marital	0.70	divorced	59	0.8983	0.0385	0.5714	0.0847
50	marital	0.70	married	288	0.9375	0.0337	0.4286	0.0729
51	marital	0.70	single	153	0.8824	0.0657	0.5625	0.1046

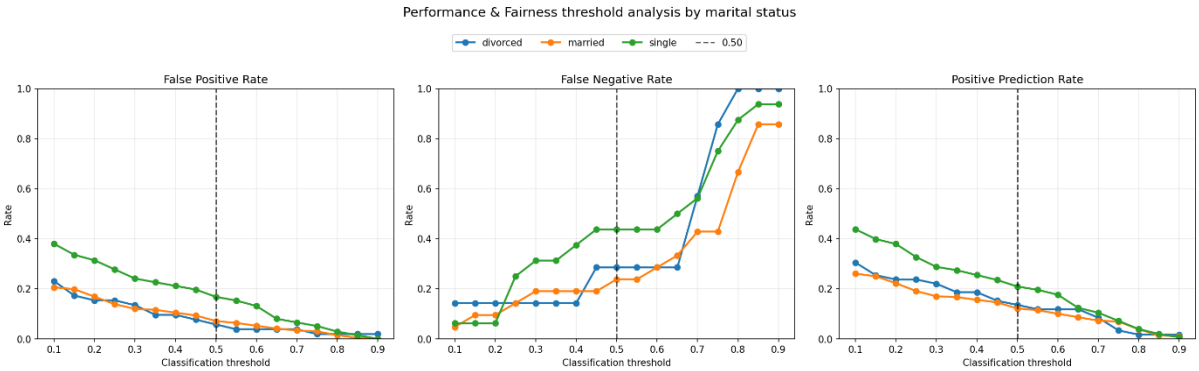


Figure 6.6: Classification-threshold analysis across marital-status groups.

6.5 Partial Dependence Analysis

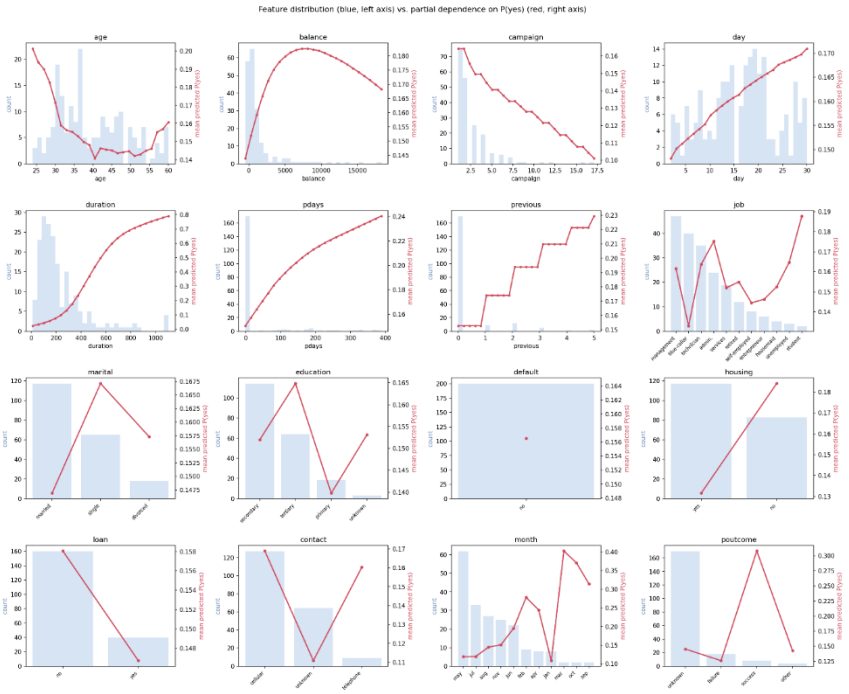


Figure 6.7: Feature distributions and partial-dependence results for the final model.

Call duration had the strongest effect: longer calls greatly increased the predicted subscription probability. Fewer campaign contacts, a successful previous outcome, and certain months were also associated with higher predictions. Unknown contact information and having housing or personal loans generally reduced the predicted probability. These results show model associations rather than causal effects, and results for rare values should be interpreted carefully.

7. Model Blessing and Deployment

The Evaluator confirmed that the final model satisfied the required BinaryAccuracy threshold and created a BLESSED artifact. The Pusher then copied the approved model to the serving-model directory, making it available for deployment. The final Pusher run created artifact /58 and confirmed that the model was successfully pushed.

```
# Blessing check + a printed summary of the model this run produced
blessing_uri = evaluator.outputs["blessing"].get()[0].uri
blessed = os.path.isfile(os.path.join(blessing_uri, "BLESSED"))
print("Blessing artifact URI:", blessing_uri)
print("Model blessed:", blessed)
if not blessed and os.path.isfile(os.path.join(blessing_uri, "NOT_BLESSED")):
    print("Reason: did not clear the BinaryAccuracy threshold (>= 0.70, or did not beat baseline by >= -0.01).")
```

```
Blessing artifact URI:
Model blessed: True
```

Figure 7.1: Evaluator output confirming that the final model was blessed.

```
pushed_model_uri = pusher.outputs["pushed_model"].get()[0].uri
pushed = pusher.outputs["pushed_model"].get()[0].get_custom_property("pushed")
print("Pushed model artifact URI:", pushed_model_uri)
print("Was the model actually pushed to the serving dir?:", bool(pushed))
if bool(pushed):
    print("Serving model directory:", SERVING_MODEL_DIR)
else:
    print("Not pushed -- model was not blessed by the Evaluator (see Step 8 output above).")
```

```
Pushed model artifact URI:
Was the model actually pushed to the serving dir?: True
Serving model directory:
```

Figure 7.2: Pusher output confirming that the blessed model was exported to the serving directory.

8. Final Reflection

8.1 What did ExampleValidator catch that raw-data inspection would have missed?

ExampleValidator found no anomalies, but this clean result was still valuable. It automatically compared the training and evaluation data with the inferred schema and confirmed that both splits were consistent with the expected features, data types, and categorical domains. It also verified that the CSV preparation step did not introduce missing or malformed values. Unlike a one-time manual inspection, this validation can be repeated automatically whenever new data enters the pipeline.

8.2 How did TFMA slicing reveal something overall accuracy alone would not?

The final model achieved an overall accuracy of 0.8791, but TFMA showed differences across marital-status groups. The single group had the lowest accuracy (0.8595) and AUC (0.9043), while the married group had the lowest precision (0.4561). The divorced group performed well but contained only 1,804 examples, making its estimates less stable than those of the larger groups. These findings demonstrate how acceptable overall results can hide subgroup-level weaknesses.

8.3 What did the What-If Tool reveal about individual predictions?

The contrasting-example comparisons showed that predictions differed when several customer and campaign features changed together. Important differences included call duration, balance, job, education, contact method, and previous campaign history. Because multiple features differed simultaneously, these comparisons do not isolate a causal effect or constitute controlled one-feature counterfactuals.

The partial-dependence results showed that call duration strongly influenced the probability of subscription. Fewer campaign contacts, a successful previous outcome, and certain campaign months were also associated with higher predictions. However, these results show model associations rather than causal relationships.

8.4 What would we change about the pipeline for production deployment?

- Remove duration if the model will be used before calling a customer, because this value is unavailable at prediction time.
- Add stronger overall and slice-level blessing thresholds so that subgroup weaknesses are considered before deployment.
- Schedule Airflow runs and configure alerts for data anomalies, failed tasks, and models that are not blessed.

- Monitor data drift and performance across additional or intersectional fairness slices.
- Use early stopping and further hyperparameter tuning to improve training stability.
- Create model documentation describing the features, intended use, known risks, and performance limitations.

8.5 Limitations of the current model and training data

The dataset is strongly imbalanced, so accuracy alone may overstate performance. Although the final model achieved a recall of 0.7144, its precision was only 0.4864, meaning that fewer than half of its positive predictions were correct.

Call duration may introduce data leakage because it is known only after a call has occurred. The dataset also comes from one Portuguese bank, so the results may not generalize to other banks, countries, or periods. Marital status represents only one fairness dimension, and additional or intersectional slices may reveal other performance differences. The What-If analysis used only 500 evaluation examples, so its findings should also be interpreted cautiously. Finally, the Pusher exported the model as a SavedModel, but the project did not deploy it as a live prediction service.

9. Conclusion

Group 6 developed and tested a complete TFX pipeline for the Bank Marketing dataset using an interactive notebook and an Airflow DAG. The pipeline covered data ingestion, statistics generation, schema creation, anomaly detection, feature transformation, model training, evaluation, model blessing, and export.

StatisticsGen, SchemaGen, and ExampleValidator confirmed that the prepared training and evaluation data followed the expected structure and contained no detected anomalies. They also revealed important concerns, including class imbalance, many unknown categorical values, and possible data leakage from call duration.

The final Model 2 achieved an overall TFMA accuracy of 0.8791, AUC of 0.9145, precision of 0.4864, and recall of 0.7144 across 15,088 evaluation examples. Compared with Model 1, it improved accuracy, AUC, and precision, although recall decreased slightly. TFMA slicing identified performance differences between marital-status groups, while the What-If Tool showed how individual features affected predictions. The Evaluator blessed the final model, and the Pusher successfully exported it to the serving-model directory.

Overall, the project demonstrated that a reliable machine-learning pipeline requires more than a strong overall accuracy value. Data validation, subgroup evaluation, model comparison, interpretability, and deployment checks are all necessary for understanding a model's performance and limitations.

10. Assumptions

- The default 2:1 training and evaluation split produced by CsvExampleGen was used without modification.
- The target label y was converted from yes and no to 1 and 0 during the one-time CSV preparation step. This was treated as acceptable data preparation before ingestion into the TFX pipeline.

- Marital status was selected as the fairness-related slicing feature because it is relevant to banking and marketing analysis. It does not represent a complete fairness assessment.
- The interactive walkthrough used 100 training steps and 20 evaluation steps, while the full Airflow pipeline used 1,000 training steps and 200 evaluation steps.
- Two model configurations were compared, but a complete automated hyperparameter search was outside the project scope.
- The Airflow Evaluator required BinaryAccuracy of at least 0.70 and allowed no more than a 0.01 decrease compared with the baseline model. These thresholds were selected for the project demonstration rather than from a formal business requirement.
- The What-If Tool used 500 randomly sampled evaluation examples. This was considered sufficient for exploratory analysis without loading the complete evaluation split into the browser widget.
- A successfully pushed SavedModel was treated as deployment preparation. The project did not create or operate a live prediction API or production service.

11. References

Moro, S., Rita, P., & Cortez, P. (2014). Bank Marketing [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5K306

TensorFlow. (n.d.). *TensorFlow Extended (TFX) documentation*. https://www.tensorflow.org/tfx

TensorFlow. (n.d.). *TensorFlow Model Analysis*. https://www.tensorflow.org/tfx/guide/tfma

PAIR-code. (n.d.). *What-If Tool documentation*. https://pair-code.github.io/what-if-tool/

Apache Software Foundation. (n.d.). *Apache Airflow documentation*. https://airflow.apache.org/docs/

Centennial College, School of Engineering Technology and Applied Science. (2026). *COMP315 term project assignment brief*.

Appendix 1: Meeting Register

Date	Time	Attendees	Subjects discussed	Assignments / action items
Jul 28, 2026	6:00 PM	All members	Kickoff — reviewed project brief, assigned dataset, agreed on roles	Set up shared repo; each member installs TFX/Airflow environment
Jul 31, 2026	7:00 PM	All members	Reviewed data prep script and CsvExampleGen output; discussed split ratio	Build StatisticsGen/SchemaGen/ExampleValidator steps
Aug 3, 2026	6:30 PM	All members	Reviewed schema anomalies (none found); discussed "unknown" category handling	Draft preprocessing_fn and run_fn module files
Aug 6, 2026	7:00 PM	All members	Reviewed first trained model and training instability; discussed class weighting	Configure EvalConfig slicing and Resolver/Evaluator/Pusher
Aug 8, 2026	6:00 PM	All members	Reviewed TFMA slice results and Fairness Indicator; chose marital as final slice	Set up Airflow DAG; run pipeline via AirflowDagRunner
Aug 10, 2026	7:30 PM	All members	Reviewed What-If Tool counterfactuals and PDPs; assigned report sections	Draft final report and reflection; rehearse presentation